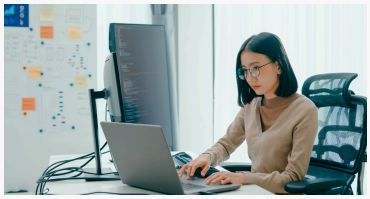
Desenvolvimento para dispositivos móveis

UNIDADE 04

Interfaces imperativas: Componentes que usam AdapterView

Olá. Anteriormente, começamos a criar interfaces gráficas com componentes simples, aquele que são comuns a todos os aplicativos. Como foi dito, aprendendo a trabalhar com um componente, o funcionamento dos demais são análogos.

Seguindo o mesmo entendimento, nesta semana vamos trabalhar com componentes um pouco mais complexos, que exigem uma fonte de organização de dados, chamadas *adapters*. Da mesma forma que ocorre com



componentes simples, vendo um desses componentes que usam *adapter*, já teremos uma noção de como funcionam os demais, pois o uso é análogo.

©Tirachard/Adobe Stock

Mãos à obra?

Videoaula: Interfaces imperativas e listas dinâmicas com RecyclerView no Android!

Nesta aula vamos criar um projeto fazendo uso de componentes um pouco mais complexos de interface gráfica imperativa. Esses componentes usam fontes de organização de dados, e são chamados de *adapters*. Os *adapters* possuem um conjunto de funções padrão destinadas a receber os dados a serem exibidos e os retorna de forma organizada para formar a estrutura visual do componente.

Para exemplificar o uso desse tipo de componente, vamos criar uma aplicação que faz um **CRUD**. CRUD é uma aplicação que cadastra um determinado tipo de informação, contendo quatro ações básicas: cadastrar (C – ***create***), visualizar ou ler (R – ***read***), editar ou atualizar (U – ***update***) e excluir ou deletar (D – ***delete***). O projeto será implementado tomando por base uma ***Recycler View***, um dos componentes mais importante de interface gráfica no desenvolvimento Android.



DICA

Para acompanhar o professor durante a explicação e exercitar o projeto, disponibilizamos, a seguir, o arquivo zipado dos códigos.

Códigos da Aula 04

Componentes de interface imperativa avançados: adapter views

Adapter views são componentes de interface gráfica imperativa em Android que permitem o gerenciamento de coleção de dados, atuando como ponte entre a fonte de dados a ser exibida e a interface gráfica do aplicativo.

São exemplos deste tipo de componentes:

- *RecyclerView.*
- *ListView.*
- *GridView.*
- *Spinner.*
- *ViewPager.*
- *ExpandableListView.*

O uso de ***adapter views*** permite um aumento de *performance* na aplicação, uma vez que realiza reaproveitamento de ***views***. Essas ***views*** são conectadas de forma dinâmica com os dados provenientes da fonte de dados da aplicação. Ao reaproveitar uma ***view***, sua interface gráfica já está criada, bastando atualizar o dado a ser exibido.

ListView x RecyclerView

ListView foi por muito tempo o componente padrão do Android para exibir listas de itens. Simples de implementar, funciona bem para listas verticais básicas. A grande dificuldade de trabalhar com *ListView* se dá pela falta de reaproveitamento de ***views***, o que reduz significantemente sua *performance* quando está trabalhando com uma massa maior de dados a serem exibidos. A ***ListView*** também tem limitações significativas: ela só funciona verticalmente, não possui suporte nativo para animações entre mudanças de dados e não é muito flexível em termos de customização, já que a maior parte da aparência e comportamento precisa ser manualmente implementada no *layout* de cada item.

Visando a suprir as necessidades do programador, sobretudo em questões de *performance*, na versão Android Support v7 (atual ***Jetpack***) foi incluído o componente ***RecyclerView***. Ela foi projetada para ser mais moderna, eficiente e extensível. Enquanto a ***ListView*** tem um comportamento mais fixo, a ***RecyclerView*** é altamente flexível: ela pode exibir listas verticais, horizontais, grids, grades irregulares e até *layout*s completamente personalizados, graças ao uso de diferentes ***LayoutManagers***. A reutilização de ***views*** é uma parte central de sua arquitetura e é implementada de forma muito mais robusta, com o uso obrigatório do padrão ***ViewHolder***. Isso faz com que sua *performance* seja superior, especialmente em listas longas.

Outra diferença importante está na forma como a ***RecyclerView*** lida com atualizações. Ela oferece métodos específicos para inserir, remover ou alterar itens individualmente, gerando animações automáticas e eficientes. Além disso, a ***RecyclerView*** suporta recursos avançados como arrastar e soltar, deslizamento para ações rápidas, decorações personalizadas entre itens e animações extensíveis, tudo por meio de classes auxiliares.

Embora seja muito mais poderosa, a ***RecyclerView*** também é mais complexa de configurar. Ela exige mais código, mais classes e mais abstrações. Por isso, a ***ListView*** ainda pode ser útil em casos extremamente simples, em que o objetivo é apenas exibir uma lista vertical sem interações especiais ou grande volume de dados. Porém, para qualquer aplicação moderna que precise de flexibilidade, *performance* e personalização, a ***RecyclerView*** é praticamente sempre a melhor escolha e é hoje o padrão recomendado pelo Android.

O padrão *ViewHolder*

O padrão ***ViewHolder*** é uma técnica criada para melhorar o desempenho de componentes de lista no Android, especialmente quando lidamos com listas longas e itens complexos. Ele surgiu inicialmente como uma recomendação para otimizar o uso de ***ListView***, mas tornou-se obrigatório na ***RecyclerView*** devido à sua importância no ciclo de vida e na eficiência da renderização.

O uso de ***ViewHolder*** evita chamadas repetidas ao método ***findViewById()***, uma operação relativamente custosa que interliga código com interface gráfica, principalmente quando ocorre centenas ou milhares de vezes durante o rolamento de uma lista. Quando utilizamos um ***ListView***, cada item exibido na tela precisa ser construído ou reutilizado. Sem o padrão ***ViewHolder***, toda vez que uma ***view*** reciclada era utilizada novamente, seria necessário percorrer toda a hierarquia de componentes do *layout* para encontrar cada ***subview*** — o título, a descrição, uma imagem, um botão — causando reconstrução de *layout*s desnecessários e desperdício de processamento.

Com o ***ViewHolder***, essa estrutura muda. Em vez de buscar as ***views*** sempre que um item é exibido, o programador cria uma classe que armazena as referências às ***subviews*** do item. Quando o *layout* é inflado pela primeira vez, essas referências são atribuídas e guardadas dentro do ***ViewHolder***. A partir daí, sempre que uma ***view*** for reutilizada, basta acessar diretamente essas referências já armazenadas, sem precisar procurar novamente na árvore de *layout*. Isso reduz drasticamente o custo de atualização de tela e torna a rolagem mais suave.

O padrão ***ViewHolder*** é parte integrante da arquitetura de uso da ***RecyclerView***. A classe ***RecyclerView.ViewHolder*** foi criada para formalizar essa prática. Cada item da lista possui seu próprio ***ViewHolder***, e a ***RecyclerView*** é responsável por gerenciar esses objetos, reciclando-os automaticamente conforme o usuário rola a lista. Isso significa que cada vez que um item precisa ser exibido, a ***RecyclerView*** não recria a ***view***, mas reaproveita um ***ViewHolder*** existente, chamando apenas o método ***onBindViewHolder()*** do ***AdapterView*** para atualizar seus dados. Esse mecanismo torna o fluxo mais eficiente e modular, além de separar melhor as responsabilidades: o ***ViewHolder*** cuida das referências de *layout*, enquanto o ***AdapterView*** cuida dos dados.

O padrão ***ViewHolder***, portanto, representa muito mais que um truque de otimização. Ele é um elemento fundamental para garantir desempenho e escalabilidade nas interfaces baseadas em listas do Android. Além de reduzir o trabalho do processador ao evitar buscas repetidas por ***views***, também promove uma arquitetura mais organizada, limpa e fácil de manter. Com ele, listas ficam mais rápidas, animadas e preparadas para lidar com grandes quantidades de dados — motivo pelo qual é uma das práticas mais importantes no desenvolvimento Android moderno.

O padrão *AdapterView*

Embora o termo ***AdapterView*** fosse tradicionalmente associado a componentes como ***ListView***, ***GridView*** e ***Spinner***, a ***RecyclerView*** não herda de ***AdapterView*** e não faz parte dessa hierarquia clássica. Ainda assim, ela utiliza o mesmo princípio de arquitetura do padrão ***AdapterView***: a separação entre dados, lógica de apresentação e as ***views*** que exibem esses dados. Na ***RecyclerView***, essa separação foi reformulada e ampliada, tornando-se mais robusta, modular e flexível do que nos componentes anteriores. Assim, mesmo sem ser literalmente uma "***AdapterView***", a ***RecyclerView*** representa a evolução desse padrão.

No modelo original de ***AdapterView***, havia três entidades principais: a ***View*** que exibia a lista, o ***Adapter*** que transformava dados em ***Views*** e o mecanismo interno de reaproveitamento. Esses componentes trabalhavam juntos, mas eram rígidos e pouco extensíveis. A ***RecyclerView*** reestruturou esse paradigma em estruturas com responsabilidades bem definidas: o ***Adapter*** continua sendo responsável por transformar dados em elementos visuais, mas agora trabalha de forma integrada com o ***ViewHolder***, que encapsula as referências de *layout*, e com o ***LayoutManager***, que define como esses itens são organizados visualmente (como lista vertical, horizontal, grid ou *layout*s customizados). A existência desses três elementos — ***Adapter***, ***ViewHolder*** e ***LayoutManager*** — torna a ***RecyclerView*** muito mais flexível e mais alinhada a boas práticas de arquitetura, reforçando um padrão semelhante ao antigo ***AdapterView***, porém expandido para novas necessidades.

Dentro do contexto da ***RecyclerView***, o ***adapter*** desempenha um papel central na aplicação desse padrão. Ele não apenas cria as ***views*** por meio do método ***onCreateViewHolder***, como também recebe um ***ViewHolder*** já existente no método ***onBindViewHolder***, em que associa os dados de forma adequada. Essa abordagem, combinada ao ciclo interno de reciclagem, expressa a essência do padrão ***AdapterView***: o ***adapter*** é a ponte entre o modelo de dados e a interface visual. A ***RecyclerView***, porém, trata essa ponte com muito mais rigor e eficiência. Em vez de apenas entregar uma ***View*** pronta, ele exige a criação de um ***ViewHolder***, que separa de forma clara os elementos de interface da lógica do ***adapter***. Essa obrigatoriedade evita erros comuns do padrão anterior e melhora muito a *performance*.

O ***LayoutManager*** da ***RecyclerView*** representa outra grande evolução em relação ao ***AdapterView*** tradicional. Nas antigas ***AdapterViews***, a forma de apresentação era fixa: o ***ListView*** só podia exibir uma lista vertical; o ***GridView*** só podia organizar itens em grade. O ***RecyclerView*** rompe com essa limitação, delegando o controle de posicionamento e

comportamento ao ***LayoutManager***. Isso significa que o padrão ***AdapterView*** deixa de ser apenas uma relação "lista-adapter" e passa a ser um sistema modular, no qual o ***Adapter*** fornece dados, o ***ViewHolder*** gerencia a ***View*** e o ***LayoutManager*** controla a apresentação. Esse desacoplamento é fundamental para a versatilidade da ***RecyclerView***.

O mecanismo de reciclagem da ***RecyclerView*** também representa uma evolução importante do padrão. Nos ***AdapterViews*** clássicos, a reciclagem existia, mas era rudimentar. Já na ***RecyclerView***, o conceito é institucionalizado. Ela possui um sistema interno de *pool* de ***ViewHolders***, responsável por armazenar diferentes tipos de *layout*s reutilizáveis, respeitando o tipo de item e garantindo que o ***adapter*** receba sempre a estrutura visual apropriada. Esse processo é mais inteligente e previsível, além de possibilitar animações, remoção incremental de ***views*** e atualizações parciais — algo inexistente no padrão ***AdapterView*** original.

Assim, quando falamos do "padrão ***AdapterView***" aplicado à ***RecyclerView***, estamos nos referindo não a uma herança literal, mas a uma evolução de arquitetura. A ***RecyclerView*** representa a maturidade desse padrão, tornando a relação entre dados e interface mais modular, eficiente e expansível. Em vez de um único componente que controla tudo, a ***RecyclerView*** trabalha com um ecossistema de classes (***adapter***, ***ViewHolder***, ***LayoutManager***) que, juntas, formam a versão moderna e mais poderosa do antigo ***AdapterView***.



EXEMPLO

Exemplo de implementação

Para entender um pouco mais sobre como funciona na prática esse padrão, além da videoaula desta semana que implementa um CRUD usando ***RecyclerView***, vamos criar um exemplo explicando cada item da arquitetura ***AdapterView***.



DOCUMENTOS

Para acompanhar as explicações do exemplo, acesse o arquivo a seguir.

Em breve.

Incialmente, vamos entender os dados com os quais vamos trabalhar: pessoas. Neste exemplo, vamos cadastrar pessoas, registrando algumas informações comuns. Para isso vamos utilizar como base de nossos dados uma classe chamada ***Pessoa***, como segue:

```
</>
```

```
1  data class Pessoa(
2      val cpf: String,
3      val nome: String,
4      val idade: Int,
5      val cidadeNascimento: String
6  )
```

A classe pessoa é uma ***data class***, um tipo de classe específico para trabalhar apenas com dados (conforme visto na semana anterior). Essa classe vai registrar os dados específico de cada objeto pessoa: CPF, nome, idade e cidade de nascimento.

Estes dados serão exibidos em uma ***RcyclerView***, como sendo uma célula de uma tabela. Essa célula terá um *layout* específico, que será criado em ***XML*** e integrado por meio do ***adapter*** aos dados de cada pessoa.

```
</>
```

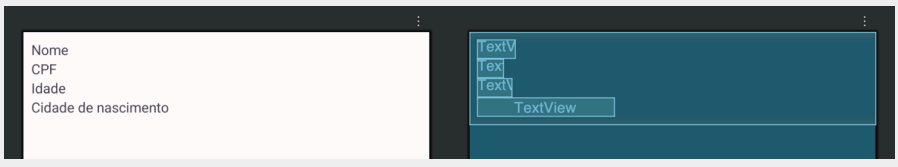
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.a
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="8dp">

    <TextView
        android:id="@+id/txtNome"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Nome" />

    <TextView
        android:id="@+id/txtCpf"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="CPF" />
```

Esse *layout*, para o qual dei o nome de ***adapter_pessoa.xml***, terá uma visualização simples dos dados, conforme apresentado o seu conteúdo visual a partir do ***Android Studio***:

#ParaTodosVerem



Fonte: O autor

Essa estrutura visual vai se repetir dentro da *RecyclerView* para todas as células da lista que tiverem informação a ser exibida.

Tendo o *layout* dos dados passamos a construir o nosso padrão *adapter*:

```
</>
```

```
1  class PessoaAdapter(
2      private val pessoas: List<Pessoa> // veto
3  ) : RecyclerView.Adapter<PessoaAdapter.Pessoa
4
5      // ViewHolder: guarda as referências das
6      class PessoaViewHolder(itemView: View) :
7          val txtNome: TextView = itemView.find
8          val txtCpf: TextView = itemView.findV
9          val txtIdade: TextView = itemView.fin
10         val txtCidadeNascimento: TextView = i
11     }
12
13     // Infla o layout do item e cria o ViewHo
14     override fun onCreateViewHolder(parent: V
15         val view = LayoutInflater.from(parent
16             .inflate(R.layout.adapter_pessoa,
17         return PessoaViewHolder(view)
18     }
```

A classe ***PessoaAdapter*** estende da classe ***RecyclerView.Adapter***, tendo por base dos dados uma classe interna chamada ***PessoaViewHolder***.

Como pode ser visto em seu construtor primário, os dados registrados são passados à classe por uma lista de objetos do tipo ***Pessoa*** (linha 2). Estes dados serão usados para popular o *layout* ao longo da execução do aplicativo.

Na linha 6, é implementado o padrão ***ViewHolder***, que recebe o *layout* criado anteriormente como base para construção de sua estrutura de dados. Assim, cada item do *layout* será representado internamente por uma ***view*** dentro de ***PessoaViewHolder***.

Na linha 14, o método ***onCreateViewHolder*** infla o *layout* criado ***adapter_pessoa.xml***, e cria o ***ViewHolder*** passando a ela o *layout* inflado. Neste método existe um parâmetro ***viewType*** que permite definir mais de um *layout* utilizado na mesma ***RecyclerView***, dando a possibilidade de apresentar diferentes tipos de celula em uma mesma listagem.

A linha 21 apresenta o método ***getItemCount***, que faz uso da lista de pessoas para retornar quantas células a ***RecyclerView*** vai ter que gerenciar.

Na linha 24, o método ***OnBindViewHolder*** recebe como parâmetros o ***holder*** e a posição do item a ser exibido. A posição vai permitir recuperar da lista de pessoas os dados que devem ser exibidos na célula em questão, e o ***holder*** para ligar estes dados com cada um dos campos do *layout*.

Por fim, temos que relacionar o ***adapter*** e o tipo de *layout* com a ***RecyclerView***. Isto é feito na ***activity*** da aplicação, a qual deve passar os dados que serão visualizados para o ***adapter*** em forma de lista, como visto anteriormente.

```
</>
```

```
1  class MainActivity : AppCompatActivity() {
2
3      private lateinit var recyclerView: Recycl
4
5      override fun onCreate(savedInstanceState:
6          super.onCreate(savedInstanceState)
7          enableEdgeToEdge()
8          setContentView(R.layout.activity_main
9          ViewCompat.setOnApplyWindowInsetsList
10             val systemBars = insets.getInsets
11             v.setPadding(systemBars.left, sys
12             insets
13         }
14
15         recyclerView = findViewById(R.id.recy
16         recyclerView.layoutManager = LinearLa
17
18         // Exemplo de vetor/lista de pessoas
```

Na linha 15, a ***RecyclerView*** é recuperada do *layout* da ***activity***, sendo atribuída à sua variável declarada na linha 3.

Na linha 16, o *layout* é atribuído, sendo que o *layout* padrão do ***LinearLayoutManager*** é o vertical, fazendo com que as células fiquem dispostas verticalmente na lista.

Os dados são criados em uma lista na linha 19, atribuindo valores que satisfaçam as exigências da classe ***Pessoal*** anteriormente definida.

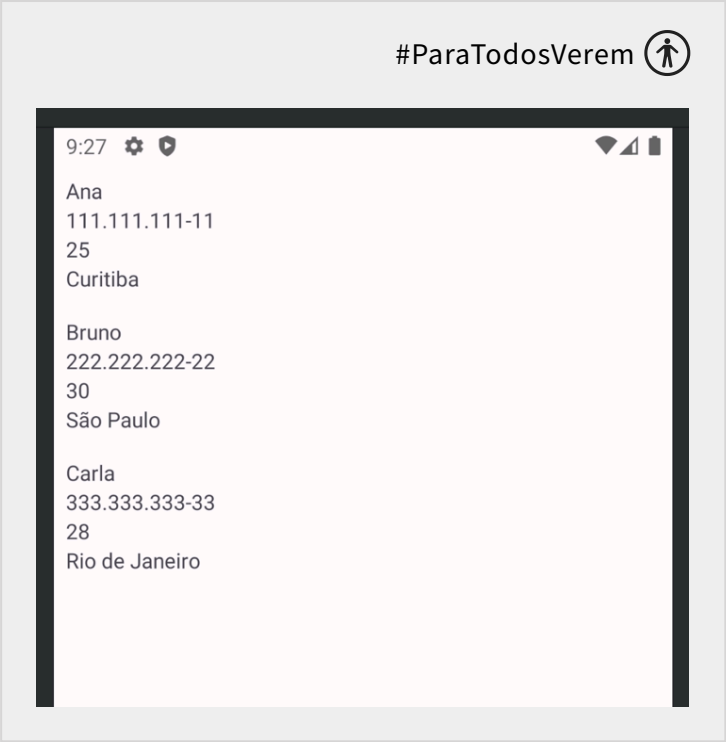
Por fim, na linha 25 o ***adapter*** baseado em ***PessoaAdapter*** é criado, recebendo a lista de pessoas como dados de origem, e é atribuído à ***RecyclerView*** na linha 26.

Segue o *layout* que foi utilizado para a ***activity*** da aplicação ***main_activity.xml***:

```
</>
```

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.andr
3      android:layout_width="match_parent"
4      android:layout_height="match_parent"
5      android:orientation="vertical"
6      android:padding="8dp"
7      android:id="@+id/main">
8
9      <androidx.recyclerview.widget.RecyclerView
10         android:id="@+id/recyclerViewPessoas"
11         android:layout_width="match_parent"
12         android:layout_height="match_parent"
13         android:scrollbars="vertical" />
14
15 </LinearLayout>
```

E assim será o resultado visual dessa aplicação. Simples, porém com vários conceitos implementados:



Fonte: O autor

Para complementar os estudos da semana, a seguir, temos um quadro mostrando diferenças entre o uso de *ListView* (padrão antigo) com *RecyclerView* e seu padrão *ViewHolder* de arquitetura.

Aspecto	***ListView***	***RecyclerView***	**Vantagens da *RecyclerView***
Estrutura	Componente antigo, rígido, focado só em lista vertical.	Componente moderno, projetado para ser modular e extensível.	Código mais organizado, fácil de evoluir e manter.
Layout suportado	Apenas lista vertical.	Lista vertical, horizontal, grid, staggered grid e *layout*s custom via ***LayoutManager.***	Um único componente resolve vários tipos de listagem.
Reutilização de *views*	Usa ***convertView***, mas depende de implementação manual.	Gerencia um *pool* de *ViewHolders* de forma automática.	Menos chance de erro e melhor *performance* em listas grandes.
Padrão *ViewHolder*	Opcional; se não usado, perde desempenho.	Obrigatório (*RecyclerView.ViewHolder*).	Arquitetura mais limpa e padronizada.

Animações	Não possui suporte nativo a animações de inserção/remoção.	Suporte nativo a animações de mudanças, inserções e remoções.	UI mais fluida e moderna sem muito esforço extra.
Status no ecossistema Android	Legado, mantido por compatibilidade.	Padrão moderno recomendado pela Google.	Investir em *RecyclerView* é investir na tecnologia atual e nas boas práticas da plataforma.



REFLEXÃO

Antes de finalizar seus estudos, propomos que faça uma pausa para responder algumas questões. Acompanhe!

1. Quais as vantagens em utilizar sempre os últimos padrões de componentes e arquiteturas no desenvolvimento Android?

2. Consigo estender os conceitos vistos nesta semana para os demais componentes que fazem uso do padrão de arquitetura ***AdapterView***?

3. Qual o grau de importância do componente ***RecyclerView*** no desenvolvimento de aplicativos para plataforma Android?



Sugestões de respostas!



1. Quais as vantagens em utilizar sempre os últimos padrões de componentes e arquiteturas no desenvolvimento Android?

Utilizar os últimos padrões de componentes e arquiteturas no desenvolvimento Android traz vantagens importantes como **maior desempenho, segurança e facilidade de manutenção**. Esses padrões são criados com base em problemas reais enfrentados pela comunidade e incorporam soluções mais eficientes e organizadas.
Além disso, arquiteturas modernas incentivam a **separação de responsabilidades**, tornando o código mais limpo, testável e escalável. Outro ponto relevante é a compatibilidade com novas versões do sistema Android, reduzindo riscos de obsolescência e garantindo uma melhor experiência para o usuário final.

2. Consigo estender os conceitos vistos nesta unidade para demais componentes que fazem uso do padrão de arquitetura *AdapterView*?

Sim, os conceitos vistos nesta semana podem ser estendidos para outros componentes que utilizam o padrão *AdapterView*, pois a lógica central permanece a mesma: **separar a fonte de dados da forma como ela é exibida na interface**. Ao compreender como o *adapter* atua como intermediário entre os dados e os componentes visuais, o desenvolvedor consegue aplicar esse raciocínio a diferentes cenários, adaptando apenas o tipo de *layout* e o comportamento da lista ou grade. Isso reforça a reutilização de código e facilita a aprendizagem de novos componentes da plataforma Android.

3. Qual o grau de importância do componente *RecyclerView* no desenvolvimento de aplicativos para plataforma Android?

O *RecyclerView* possui um grau de importância **muito elevado** no desenvolvimento de aplicativos Android, pois é o componente padrão para exibição de listas e coleções de dados de forma eficiente.
Ele oferece melhor desempenho em comparação a componentes mais antigos, como o *ListView*, ao reutilizar elementos visuais e reduzir o consumo de memória. Além disso, o *RecyclerView* é altamente flexível, permitindo diferentes tipos de *layout*, animações e interações, o que o torna essencial para a criação de interfaces modernas, responsivas e alinhadas às boas práticas do ecossistema Android.

Ligando conceitos

Adotar o ***RecyclerView*** como recomendado pela Google é essencial para manter aplicações Android modernas, eficientes e alinhadas às boas práticas da plataforma. Diferente do ***ListView***, o ***RecyclerView*** exige o uso do padrão ***ViewHolder***, garantindo melhor desempenho ao eliminar buscas repetidas por ***views*** e tornando o código mais organizado e fácil de manter. Sua arquitetura modular — separando ***Adapter***, ***ViewHolder*** e ***LayoutManager*** — oferece flexibilidade e *performance* superiores, além de integrar recursos atuais como animações, atualização inteligente de listas e suporte a diferentes tipos de *layouts*. Seguir essas recomendações proporciona interfaces mais responsivas, escaláveis e preparadas para a evolução constante do ecossistema Android.

Referências

DESENVOLVEDORES ANDROID. Disponível em: https://developer.android.com/?hl=pt-br. Acesso em: 24 out. 2025.

JETBRAINS. **Kotlin Programming Language**. Disponível em: https://kotlinlang.org/. Acesso em: 24 out. 2025.

© PUCPR - Todos os direitos reservados.